

Hutool如何使用Builder模式创建线程池

前言

Builder 设计模式也叫做 **构建者模式或者建造者模式**，名字只是一种叫法，当聊起三种名称的时候知道是怎么回事就行。

Builder 设计模式在作者编码过程中，**属于比较常用的模式之一**。优秀的设计模式总是会受到广大开发者的青睐，Hutool 也是其中之一。

因为上周编写的业务需要用到线程池，就去 Hutool thread 包下看了看，还真有惊喜，学习到了一种之前编码中没用过的 Builder 模式实现。

这里必须提一句：**设计模式重要的是思想**，一种设计模式可能不止一种实现方式。

Builder 模式应用场景

Builder 模式作用域：**如果类的属性之间有一定的依赖关系或者约束条件（源自设计模式之美）**，那么就可以考虑使用 Builder 设计模式。

我们依照线程池来举例，默认创建的线程池，构造方法最多有七个参数，核心线程数、最大线程数、阻塞队列、线程存活时间...

日常使用创建线程池时，大家想一下为什么要这么设计？一起来看下源码注释中如何解释此行为。

```

* @param handler the handler to use when execution is blocked
*         because the thread bounds and queue capacities are reached
* @throws IllegalArgumentException if one of the following holds:<br>
*         {@code corePoolSize < 0}<br>
*         {@code keepAliveTime < 0}<br>
*         {@code maximumPoolSize <= 0}<br>
*         {@code maximumPoolSize < corePoolSize}
* @throws NullPointerException if {@code workQueue}
*         or {@code threadFactory} or {@code handler} is null
*/
public ThreadPoolExecutor(int corePoolSize,
                           int maximumPoolSize,
                           long keepAliveTime,
                           TimeUnit unit,
                           BlockingQueue<Runnable> workQueue,
                           ThreadFactory threadFactory,
                           RejectedExecutionHandler handler) {

```

线程池之所以设置如此之多的构造参数，是因为对这些参数会有一定规则的校验，如果不满足线程池的规则，将不允许创建线程池，通过抛异常的方式终止程序。

终止规则大概有七点，这里列举一下：

1. 核心线程数不可以小于 0。
2. 线程存活时间不可以小于 0。
3. 最大线程数不可以小于等于 0，同时也不可以小于核心线程数。
4. 阻塞队列、线程工厂、拒绝策略参数均不可为空。

上述七点有两个作用，其一是为了让核心参数满足线程池运行流程，其二是为了保障运行时的稳定性。

小伙伴想一哈线程池创建是不是灰常灰常适合 Builder 模式，构造器函数过多以及属性之间存在依赖关系和约束条件。

Hutool Builder 创建线程池

Hutool 线程池相关使用 Builder 设计模式有两处，一个是创建线程池，另一个是创建线程工厂，我们重点围绕线程池说。

创建 Hutool 线程池比较简单且优雅，笔者较喜欢这种链式风格，所以抽象公共业务时都会使用此模式，如图所示。

```

public ExecutorService buildWriteTaskPool() {
    ThreadPoolExecutor executor = ExecutorBuilder.create()
        // 核心线程数
        .setCorePoolSize(5)
        // 最大线程数
        .setMaxPoolSize(5 << 1)
        // 阻塞队列
        .setWorkQueue(new LinkedBlockingQueue(500))
        // 线程存活时间
        .setKeepAliveTime(30, TimeUnit.SECONDS)
        // 线程池拒绝策略
        .setHandler(new ThreadPoolExecutor.AbortPolicy())
        // 线程工厂
        .setThreadFactory(ThreadFactoryBuilder.create().setNamePrefix("biz-type-").build())
        .build();
    return executor;
}

```

这个时候跟下源码，先从 `ExecutorBuilder#create` 入手，小伙伴就明白 Hutool 是如何玩 Builder 模式了。

```

public static ExecutorBuilder create() {
    return new ExecutorBuilder();
}

```

What？自己创建自己？这是要搞啥子。

小伙伴想一下，如果你想要对一个类中属性进行约束，前提是不是先应该把属性搞到手。

没错，`ExecutorBuilder#create` 方法返回自己本身，然后通过 set 方法 **把数据填充到创建出来的对象上**，最后再进行依赖关系整理和条件约束。

看一下 `ExecutorBuilder#build` 方法内部做了什么事情。

```

@Override
public ThreadPoolExecutor build() {
    return build(this);
}

/** ...
private static ThreadPoolExecutor build(ExecutorBuilder builder) {
    final int corePoolSize = builder.corePoolSize;
    final int maxPoolSize = builder.maxPoolSize;
    final long keepAliveTime = builder.keepAliveTime;
    final BlockingQueue<Runnable> workQueue;
    if (null != builder.workQueue) { ...
    } else { ...
    }
    final ThreadFactory threadFactory = (null != builder.threadFactory) ? builder.threadFactory : Executors.defaultThreadFactory();
    RejectedExecutionHandler handler = ObjectUtil.defaultIfNull(builder.handler, new ThreadPoolExecutor.AbortPolicy());

    final ThreadPoolExecutor threadPoolExecutor = new ThreadPoolExecutor(/* ...
    );
    if (null != builder.allowCoreThreadTimeOut) { ...
    }
    return threadPoolExecutor;
}

```

这里有个知识点，也是 B 格之一，大家看到 build 方法上有 @Override 注解，证明它是实现了接口方法。

```

/**
 * 建造者模式接口定义
 *
 * @param <T> 建造对象类型
 * @author Looly
 * @since 4.2.2
 */
public interface Builder<T> extends Serializable{
    /**
     * 构建
     *
     * @return 被构建的对象
     */
    T build();
}

```

Hutool 定义了 Builder 接口，实现此接口即可完成 Builder 模式，泛型 T 代表需要返回的构造对象类型，比如刚才线程池 Builder 泛型就是 ThreadPoolExecutor。

在实现 build 方法上调用真正管理依赖和约束的方法 build(ExecutorBuilder builder)，将刚才创建好并且已经赋过值的构建对象传入。

最后 build(ExecutorBuilder builder) 返回的就是我们所需要的线程池对象，这一块大家可以自己跟下源码，学会就可以套用自己写的业务代码。

Builder 模式不同的实现方式

上文说过，设计模式重思想，就像 Builder 模式，强调的是 **管理依赖关系或者约束条件**。

刚才 Hutool Builder 只是一种实现方式，之前还用过静态内部类的实现方式。

代码经过精剪，并且为了阅读体验感，把部分缩进去除了。不过笔者测试过粘贴到 IDEA 中编译是可以的。

```
@Getter
public class HttpParameters {
    private Builder builder;
    public static Builder newBuilder() { return new Builder(); }
    private HttpParameters(Builder builder) { this.builder = builder; }

    @Getter
    public static class Builder {
        private String url;
        private Object parameter;
        private String httpType;
        public Builder parameter(Object parameter) { this.parameter = parameter; return
this;}
        public Builder url(String url) { this.url = url; return this; }
        public Builder httpType(String httpType) { this.httpType = httpType; return this;
    }

        public HttpParameters build() {
            if (StringUtils.isBlank(url)) {throw new RuntimeException("URL不允许为空 ");
        }

            // ...
            return new HttpParameters(this);
        }
    }
}
```

如果后面要获取 HttpParameters 参数就需要先获取 Builder 对象。

可能有些小伙伴不习惯这种方式，也可以把 Builder 对象属性在 Parameters 里也定义一份，方式都很灵活。

文末结语

本文通过创建线程池为引，讲述了 Builder 设计模式的场景以及实际用途，并引用 Hutool Builder 模式创建线程池进行讲解。

相信大家看完之后对 Builder 模式的场景以及应用有了更深入的了解，另外我们可以将 Builder 模式引入到自己代码中，实际操练一下，相信你也会对它 "爱不释手"。

更新: 2023-07-17 15:29:20

原文: <<https://www.yuque.com/magestack/12306/qztuml34mqglvdq8>>